

```

// copy/pasta from the adafruit gps library software serial parsing example
// Test code for Adafruit GPS modules using MTK3329/MTK3339 driver
//
// This code shows how to listen to the GPS module in an interrupt
// which allows the program to have more 'freedom' - just parse
// when a new NMEA sentence is available! Then access data when
// desired.

#include <Adafruit_GPS.h>
#include <SoftwareSerial.h>

// Connect the GPS Power pin to 5V
// Connect the GPS Ground pin to ground
// Connect the GPS TX (transmit) pin to Digital 8
// Connect the GPS RX (receive) pin to Digital 7

#define GPSSerial Serial2

Adafruit_GPS GPS(&GPSSerial);

// Set GPSECHO to 'false' to turn off echoing the GPS data to the Serial
// console
// Set to 'true' if you want to debug and listen to the raw GPS sentences
#define GPSECHO false

/*
 * PIN SETUP FOR MAGNOMETER ----> just use the default and there's no
 * need to define pins (from the wire docs
 * https://docs.arduino.cc/language-reference/en/functions/communication/wire
 */
* SDA -> D20
* SCL -> D21
* GND -> GND
* VCC -> PWR
* INCLUDE QMC5883LCompass Library
*/

#include <ServoTimer2.h>
#include <Wire.h>
#include <QMC5883LCompass.h>

```

```

// Motor A (LEFT) connections
int enA = 3;
int in1 = 4;
int in2 = 5;

// Motor B (RIGHT) connections
int enB = 8;
int in3 = 6;
int in4 = 7;

//magnetometer connection
QMC5883LCompass mag;
float theta;
int x; //EAST & WEST
int y; //NORTH & SOUTH

#define rightMotorspeed 200
#define leftMotorspeed 200 //base speed (added to both motors when
driving)
#define steeringLock 40 //maximum amount of steering (can increase for
tighter turning)

//payload servo
int servoPayloadPin = 13;
int dropAngle = 500;
int holdAngle=2000;
ServoTimer2 payloadServo;

float steerGain = -0.5; //gain between compass error and steering value
(you want a negative value so positive (right) heading steers negative
(left))

float magFilter = 0.2; //alpha constant for the compass exponential
filter

float errorFilter = 0.1; //constant for the gps heading error filter

float magOffset = 30.0; //calibration offset for compass (deg)

```

```
float rawMag = 0.0; //raw compass heading

float rawMaglast = 0.0; //last reading for updating

float magHeading = magOffset; //filtered compass heading

// float gpsPosX = 0.0; //gps positions for mission 4

// float gpsPosY = 0.0;

// float gpsStartlat = 0.0; //set the start point of the mission (lat/lon
angles) and do math from there

// float gpsStartlon = 0.0;

float gpsHeading = 0.0; //raw gps heading

float gpsHeadinglast = 0.0; //stores last heading to check for updated
value

float headingError = 0.0; //an offset between the mag and gps headings.
builds up over time with a filter. positive if gps>mag

bool missionStarted = false; //variable for waiting for first gps fix
before driving

bool gpsUpdated=false; //signals that fresh gps data has come through

uint32_t timer = millis(); //timer for use in the loop

uint32_t missionStart = millis(); //timer for mission end

void setup()
{
    // Set all the motor control pins to outputs
    pinMode(enA, OUTPUT);
    pinMode(enB, OUTPUT);
    pinMode(in1, OUTPUT);
    pinMode(in2, OUTPUT);
    pinMode(in3, OUTPUT);
```

```

pinMode(in4, OUTPUT);

// Turn off motors - Initial state
digitalWrite(in1, LOW);
digitalWrite(in2, LOW);
digitalWrite(in3, LOW);
digitalWrite(in4, LOW);

payloadServo.attach(servoPayloadPin);
payloadServo.write(holdAngle);

mag.init();
mag.setCalibrationOffsets(-1332.00, -764.00, -706.00);
mag.setCalibrationScales(0.76, 1.02, 1.43);
//more adafruit example cospasta
// 9600 NMEA is the default baud rate for Adafruit MTK GPS's- some use
4800
// 9600 NMEA is the default baud rate for Adafruit MTK GPS's- some use
4800
GPS.begin(9600);
// uncomment this line to turn on RMC (recommended minimum) and GGA (fix
data) including altitude
GPS.sendCommand(PMTK_SET_NMEA_OUTPUT_RMCGGA);
// uncomment this line to turn on only the "minimum recommended" data
//GPS.sendCommand(PMTK_SET_NMEA_OUTPUT_RMCONLY);
// For parsing data, we don't suggest using anything but either RMC only
or RMC+GGA since
// the parser doesn't care about other sentences at this time
// Set the update rate
GPS.sendCommand(PMTK_SET_NMEA_UPDATE_1HZ); // 1 Hz update rate
// For the parsing code to work nicely and have time to sort thru the
data, and
// print it out we don't suggest using anything higher than 1 Hz

// Request updates on antenna status, comment out to keep quiet
GPS.sendCommand(PGCMD_ANTENNA);
}

void loop() {
    //mag reading and updates

```

```

mag.read(); //read mag
rawMag = mag.getAzimuth()+magOffset; //get raw angle reading with offset
(north is 0.0)

if(rawMag!=rawMaglast){ //only update filter if value is new.
    magHeading = magHeading + magFilter*(rawMag-magHeading); //exponential
filter for mag heading
    rawMaglast=rawMag;
}

//main driving loop
if(missionStarted){
    if(gpsUpdated){
        headingError = headingError +
errorFilter*((gpsHeading-magHeading)-headingError); //update the heading
error through the exponential filter
        gpsUpdated=false;
    }
    drive((magHeading+headingError)*steerGain);
}

//gps reading stuff
char c = GPS.read();
// if you want to debug, this is a good time to do it!
if ((c) && (GPSECHO))
    Serial.write(c);

// if a sentence is received, we can check the checksum, parse it...
if (GPS.newNMEAreceived()) {
    // a tricky thing here is if we print the NMEA sentence, or data
    // we end up not listening and catching other sentences!
    // so be very wary if using OUTPUT_ALLDATA and trying to print out
data
    //Serial.println(GPS.lastNMEA()); // this also sets the
newNMEAreceived() flag to false

    if (!GPS.parse(GPS.lastNMEA())) // this also sets the
newNMEAreceived() flag to false
        return; // we can fail to parse a sentence in which case we should
just wait for another

```

```

}

if(millis()-missionStart>40000){
    stop();
    payloadServo.write(dropAngle);
}

if (millis() - timer > 1000) {
    timer = millis(); // reset the timer
    if (GPS.fix) {
        gpsHeading = GPS.angle;
        if (gpsHeading > 180) {
            gpsHeading -= 360; //keep the gps angle -180 to 180 like the mag
        }
        // for later, these are the lat and lon variables you will need to
reference: GPS.lat GPS.lon
        if(gpsHeading != gpsHeadinglast){
            gpsHeadinglast=gpsHeading;
            gpsUpdated=true; //signal fresh data
        }
        if(!missionStarted){
            missionStart=millis();
        }
        missionStarted=true;
    }
}

}

void drive(float steer){ //steer is positive right
    // Turn on motors
    digitalWrite(in1, LOW);
    digitalWrite(in2, HIGH);
    digitalWrite(in3, HIGH);
    digitalWrite(in4, LOW);

    int ster = constrain(steer,-steeringLock,steeringLock); // constrain the
steering amount to plus/minus steeringLock

    analogWrite(enA, constrain((int)round(leftMotorspeed + steer),0,255));
//constrain and write the motor powers

```

```
    analogWrite(enB, constrain((int)round(rightMotorspeed - steer),0,255));  
}  
  
void stop() {  
    digitalWrite(in1, LOW);  
    digitalWrite(in2, LOW);  
    digitalWrite(in3, LOW);  
    digitalWrite(in4, LOW);  
    delay(500);  
}
```